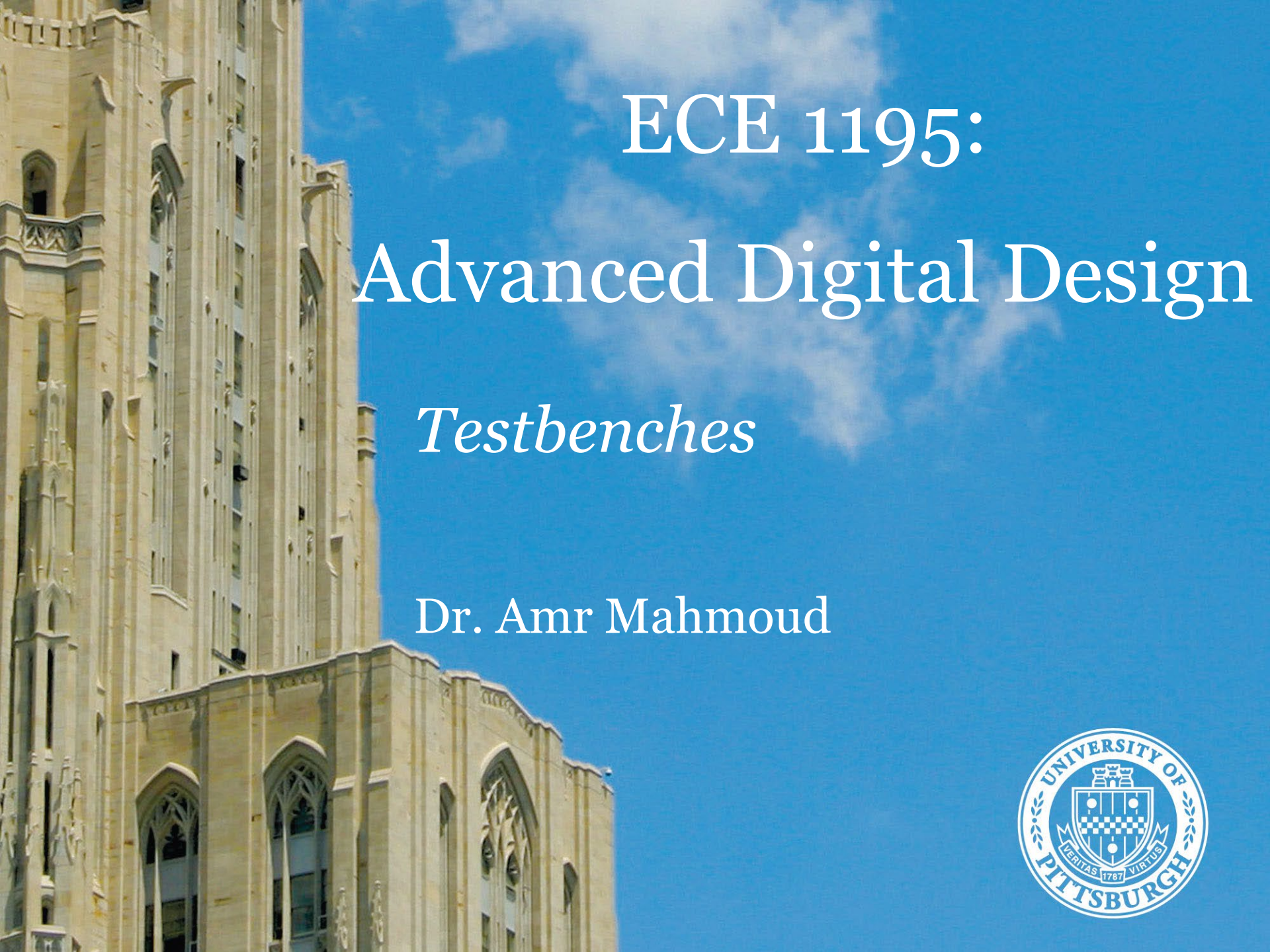


ECE 1195: Advanced Digital Design

Testbenches

Dr. Amr Mahmoud

VHDL TESTBENCHES

VHDL Testbenches

- These are modules to help in automating the test of your design functionality.
  - Usually they don't have any in/out ports (entity part is empty)
  - Are not meant to be synthesized

VHDL Testbench Example

- adder_tb.vhd (lab 1)

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity adder_tb is
end entity;

architecture sim of adder_tb is
  component adder is
    generic (
        WIDTH : positive := 8
    );
    port (
        A : in  std_logic_vector(WIDTH-1 downto 0);
        B : in  std_logic_vector(WIDTH-1 downto 0);
        S : out std_logic_vector(WIDTH-1 downto 0)
    );
  end component;

  constant WIDTH : positive := 4;
  signal A : std_logic_vector(WIDTH-1 downto 0);
  signal B : std_logic_vector(WIDTH-1 downto 0);
  signal S : std_logic_vector(WIDTH-1 downto 0);
...
```

```
...
begin
  DUT: adder
    generic map (
        WIDTH => WIDTH
    )
    port map (
        A     => A,
        B     => B,
        S     => S
    );

  process
  begin
    for i in 0 to 2**WIDTH-1 loop
        for j in 0 to 2**WIDTH-1 loop
            A <= std_logic_vector(to_unsigned(i, WIDTH));
            B <= std_logic_vector(to_unsigned(j, WIDTH));
            wait for 10 ns;
        end loop;
    end loop;
    wait;
  end process;
end architecture;
```

VHDL Testbenches

- Random Testing for the 32-bit Add/Sub in Lab 1
  - It won't be reasonable to test all the possible combination of the input for a 32-bit adder/subtractor that we have.
  - It makes more sense to divide your test procedure into 2 cases: testing extreme corner cases, and random input testing.
- Corner cases (example):
  - A + B, and the result is less than or equal to +2^32-1 (no Carry out).
  - A + B, and the result is greater than +2^32-1 (Carry out).
  - A - B is positive (Carry out).
  - A - B is zero (Carry out).
  - A - B is negative (no Carry out).
- Random testing:
  - Randomly choose 2 numbers, add (subtract) them, make sure that result is correct.

VHDL Testbenches

- Automation using `assert – report`

```
assert (Condition) report "Text Message";
```

If the condition is false, the message will be displayed and the execution of the tb will stop!

VHDL Testbenches

- Automation using `assert - report`

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity FA_tb is
6  end FA_tb;
7
8  architecture TB of fa_tb is
9
10   signal A, B, Cin, S, Cout : std_logic;
11
12   component FA is
13     port (
14       A    : in std_logic;
15       B    : in std_logic;
16       Cin  : in std_logic;
17       S    : out std_logic;
18       Cout : out std_logic
19       );
20   end component;
21
22 begin  -- TB
23   UUT : FA
24     port map (
25       A    => A,
26       B    => B,
27       Cin  => Cin,
28       S    => S,
29       Cout => Cout);
30
```

VHDL Testbenches

- Automation using `assert - report`

```
  process
    variable temp : std_logic_vector(2 downto 0);
  begin
    for i in 0 to 7 loop
      temp := std_logic_vector(to_unsigned(i,3));
      A <= temp(2);
      B <= temp(1);
      Cin <= temp(0);
      wait for 10 ns;
      assert(S = (A xor B xor Cin)) report "Sum failed";
      assert(Cout = ((A and B) or (A and Cin) or (B and Cin))) report "Carry failed";
    end loop;  -- i
    report "SIMLUATION FINISHED!";
    wait;
  end process;
end TB;
```

C/C++ TESTBENCHES

C/C++ Testbenches

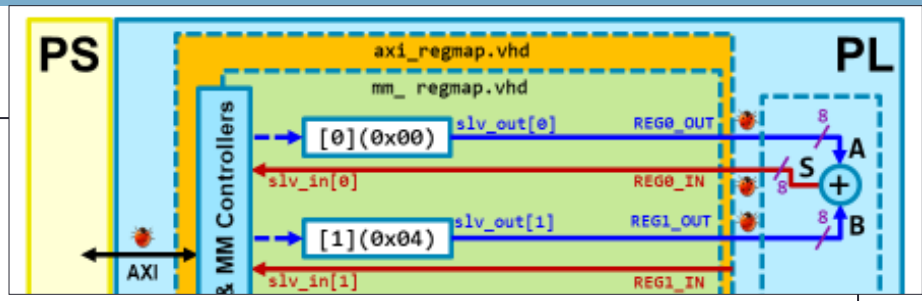
- This is a method to test your design after FPGA deployment, leveraging the ZYNQ processing system

C/C++ Testbenches

- VHDL Testbenches
  - Functional Simulation
  - Post-Synthesis (Functional/Timing) Simulation
  - Post-Implementation (Functional/Timing) Simulation
- C/C++ Testbenches
  - Post-Configuration (Deployment) Simulation

C/C++ Testbench Example

- adder.c (lab 1)




```c
#include <stdio.h>

int main(void)
{
    // base address of axi_regmap as set in the Vivado address editor
    uint32_t *regmap = (uint32_t *) 0x70000000;

    for (size_t i = 0; i < 16; i++) {
        for (size_t j = 0; j < 16; j++) {
            regmap[0] = i;          // write to register 0 (REG0_OUT, input A of adder)
            regmap[1] = j;          // write to register 1 (REG1_OUT, input B of adder)
            size_t r  = regmap[0]; // read from register 0 (REG0_IN, output S of adder)
            printf("%2zu + %2zu = %3zu (%s)\n", i, j, r, (r == i+j) ? "COR" : "ERR");
        }
    }

    return 0;
}
```

Register Map			
Index	Offset	Read Access	Write Access
[0]	0x00	S (adder output)	A (adder input)
[1]	0x04	Zero	B (adder input)